

INSTRUCTION FETCH UNIT VERIFICATION PLAN

1. Feature Checklist

The following features must be verified for functional correctness:

- **Prefetching Logic:** Proactive instruction fetching to decouple memory latency from execution.
- **Fetch FIFO:** Proper operation of the 3-entry storage buffer and its feedthrough path for zero-latency delivery when empty.
- **Memory Protocol:** Adherence to the req/gnt/rvalid handshake for all memory transactions.
- **Alignment & RVC:** Correct expansion of 16-bit compressed instructions and handling of 32-bit instructions crossing word boundaries.
- **Control Flow:** Correct flushing of the buffer on branches, jumps, and exceptions.
- **Static Branch Prediction:** Accuracy of backward-taken and forward-not-taken guesses and the associated 1–2 cycle penalties.

2. Verification Methodology

To achieve high confidence, we will use a combination of automated and directed techniques:

A. Verification Strategy

- **C++ - based Testbench:** Implement C++ in building the testbench.
- **ISS Reference Model (Spike):** Use the Spike as the reference model.
- **Co-simulation (Cross-check):** The testbench runs a program on both the **real Ibex hardware (RTL)** and the **Spike software model** at the same time. It then compares them step-by-step.

- **RISC-DV:** This is a tool that generates random, complex instructions.

B. Constrained Random Verification (CRV)

- **Memory Delays:** Randomly delay the `instr_gnt_i` and `instr_rvalid_i` signals to simulate variable memory latency.

C. SystemVerilog Assertions (SVA)

- **Protocol Integrity:** Assert that `instr_req_o` remains high until `instr_gnt_i` is asserted.
- **Alignment Check:** Assert that `instr_addr_o[1:0]` is always `2'b00` for every request.
- **FIFO Bounds:** Assert that the FIFO never overflows beyond its configurable `DEPTH`.

D. Functional Coverage

- **Cross-Coverage:** Track combinations of instruction types (Compressed vs. Uncompressed) and their alignment boundaries (Aligned vs. Misaligned).
- **Flush Conditions:** Ensure coverage of flushes triggered specifically by branches, jumps, and illegal instruction exceptions.

3. Detailed Test Scenarios

Test Case	Description	Goal
Linear Flow	Execute a straight sequence of 32-bit instructions.	Verify basic "1 instruction per cycle" throughput.
Backpressure	Stall the ID stage while memory continues to supply instructions.	Verify FIFO fills to <code>DEPTH</code> and correctly halts fetching.
Misaligned Stress	Jump to addresses ending in <code>0x2</code> or <code>0x6</code> .	Verify the IFU performs two word-aligned fetches to stitch one 32-bit instruction.
Flush-on-Branch	Execute a taken branch.	Verify current buffer contents are cleared and the "New PC" is fetched immediately.

Prediction Penalty	Force a misprediction on a forward branch.	Verify the 1-cycle or 2-cycle penalty is correctly applied based on alignment.
Zero-Latency	Deliver <code>rvalid_i</code> in the cycle immediately following <code>gnt_i</code> .	Test the "feedthrough path" efficiency.

Note: Clarify the ambiguity in the Zero-Latency test : How the “`rvalid_i`” comes right after the `gnt_i`” helps us to verify the feedthrough path.

A. Definition of Zero-Latency

In the memory protocol, the fastest possible response is:

- **Cycle 1:** The IFU asks for data (`instr_req_o`).
- **Cycle 2:** The memory accepts the request (`instr_gnt_i`).
- **Cycle 3:** The memory delivers the data (`instr_rvalid_i`).

When `instr_rvalid_i` arrives just one cycle after the grant, we call it a "Zero-Latency" response because there was no extra "wait time" from the memory.

B. Buffered vs. Feedthrough Timing

The **feedthrough path** is a "shortcut" used only when the FIFO is empty. Here is how the test reveals if that shortcut is working:

The "Slow" Buffered Way (No Feedthrough)

If the shortcut is broken, the instruction must be saved into a register first.

- **Cycle 3:** `instr_rvalid_i` is high. The instruction enters the FIFO and is saved
- **Cycle 4:** The instruction finally appears on the IFU output for the Decode stage.
- **Result:** A 1-cycle delay.

The "Fast" Feedthrough Way

If the shortcut is working and the FIFO is empty, the data doesn't wait for the next clock tick.

- **Cycle 3:** `instr_rvalid_i` is high. The instruction "feeds through" the logic.
- **Cycle 3 (Same Cycle):** The instruction is already available on the IFU output for the Decode stage.
- **Result:** Zero-cycle delay.

C. Why this verifies “Efficiency”

By delivering the `instr_rvalid_i` as fast as possible (immediately after the grant), we create a high-speed scenario. If the IFU can hand that instruction to the Decode stage in

that **exact same cycle**, it proves the feedthrough path is:

1. **Correctly built:** The data is bypassing the internal storage registers.
2. **Correctly triggered:** The logic properly detected that the FIFO was empty and opened the "shortcut".